

Aula 05

Modelos N-gram e Probabilísticos

PROCESSAMENTO DE LINGUAGEM NATURAL - IBM3130

Prof. Anderson Vanin

1. Introdução

Na Aula 04 aprendemos a representar textos como vetores de números usando **Bag of Words** e **TF-IDF**. Essas representações funcionam bem para muitas tarefas, mas têm uma limitação fundamental: elas ignoram completamente a ordem das palavras.

Para o *BoW*, as frases abaixo são idênticas:

"o gato comeu o rato"

"o rato comeu o gato"

Ambas produzem o mesmo vetor — mas têm significados completamente diferentes.

1. Introdução

Os Modelos N-gram resolvem exatamente esse problema. Em vez de olhar para palavras isoladas, eles olham para **sequências de palavras consecutivas**. Isso permite capturar contexto, prever a próxima palavra e até gerar texto.

1. Introdução

Conceito	O que é
N-grama	Sequência de n palavras consecutivas
Unigrama	Cada palavra isolada (n=1)
Bigrama	Par de palavras consecutivas (n=2)
Trigrama	Trio de palavras consecutivas (n=3)
Modelo de linguagem	Sistema que calcula a probabilidade de uma sequência de palavras
Probabilidade condicional	A chance de uma palavra aparecer dado o que veio antes
Suavização de Laplace	Técnica para tratar palavras nunca vistas
Perplexidade	Métrica que mede a qualidade de um modelo de linguagem

1.1 O que são N-gramas?

Um **n-grama** é uma sequência contígua de **n** itens de um texto. Os itens podem ser letras, sílabas ou — na maioria dos casos em PLN — palavras.

O nome vem do grego: **n** é o número de elementos da sequência, e **grama** significa letra/escrita.

1.1 O que são N-gramas?

Exemplos com a frase: "**o modelo aprende com os dados**"

- **Unigramas (n = 1) — cada palavra isolada:**

["o", "modelo", "aprende", "com", "os", "dados"]

- **Bigramas (n = 2) — pares consecutivos:**

["o modelo", "modelo aprende", "aprende com", "com os", "os dados"]

- **Trigramas (n = 3) — trios consecutivos:**

["o modelo aprende", "modelo aprende com", "aprende com os", "com os dados"]

1.1 O que são N-gramas?

Como calcular a quantidade de n-gramas

Dado um texto com T tokens, a quantidade de n-gramas possíveis é:

$$\text{quantidade} = T - n + 1$$

No exemplo acima (T = 6 tokens):

- Unigramas: $6 - 1 + 1 = 6$
- Bigramas: $6 - 2 + 1 = 5$
- Trigramas: $6 - 3 + 1 = 4$

1.2 Por que N-gramas são importantes?

Problema 1 — Negação ignorada pelo BoW

No Bag of Words, "*funciona*" e "*não funciona*" geram tokens separados. Um classificador pode confundir os dois contextos.

Com bigramas, "*não funciona*" é um único token — o contexto é preservado.

Sem bigrama: ["não", "funciona"] → dois tokens independentes

Com bigrama: ["não funciona"] → um token com contexto

1.2 Por que N-gramas são importantes?

Problema 2 — Termos técnicos fragmentados

Expressões compostas perdem o sentido quando separadas:

"aprendizado de máquina"	→	["aprendizado", "de", "máquina"]	← sem contexto
"aprendizado de máquina"	→	["aprendizado de", "de máquina"]	← com bigrama
"aprendizado de máquina"	→	["aprendizado de máquina"]	← com trigrama

1.2 Por que N-gramas são importantes?

Problema 3 — Ambiguidade de sentido

A palavra "banco" tem significados diferentes dependendo do contexto:

"fui ao banco sacar dinheiro"	→ bigrama "banco sacar"	→ contexto financeiro
"sentei no banco da praça"	→ bigrama "banco praça"	→ contexto de assento

1.3 Extrair N-gramas manualmente

```
tokens = ["o", "modelo", "aprende", "com", "os", "dados"]
```

```
bigramas = []
```

```
for i in range(len(tokens) - 1):
```

```
    par = tokens[i] + " " + tokens[i+1]
```

```
    bigramas.append(par)
```

```
print(bigramas)
```

```
# ["o modelo", "modelo aprende", "aprende com", "com os", "os dados"]
```


1.3 Extrair N-gramas manualmente

```
trigramas = []  
  
for i in range(len(tokens) - 2):  
    trio = tokens[i] + " " + tokens[i+1] + " " + tokens[i+2]  
    trigramas.append(trio)  
  
print(trigramas)  
  
# ["o modelo aprende", "modelo aprende com", "aprende com os", "com os  
dados"]
```

1.4 N-gramas com NLTK - função ngrams()

```
from nltk import ngrams
from nltk.tokenize import word_tokenize

frase = "o modelo de linguagem aprende com os dados"
tokens = word_tokenize(frase, language='portuguese')

# Gerar bigramas
bigramas = list(ngrams(tokens, 2))
print("Bigramas:", bigramas)

# Gerar trigramas
trigramas = list(ngrams(tokens, 3))
print("Trigramas:", trigramas)
```

1.5 Frequência de N-gramas

Assim como contamos palavras com FreqDist, podemos contar n-gramas:

```
from nltk import ngrams, FreqDist
```

```
from nltk.tokenize import word_tokenize
```

```
corpus = [
```

```
    "o produto chegou rápido e funciona bem",
```

```
    "o produto não funciona e não chegou",
```

```
    "chegou rápido mas não funciona direito",
```

```
]
```


1.5 Frequência de N-gramas

```
# Juntar todos os tokens do corpus
```

```
todos_tokens = []
```

```
for frase in corpus:
```

```
    tokens = word_tokenize(frase.lower(), language='portuguese')
```

```
    for token in tokens:
```

```
        todos_tokens.append(token)
```

```
# Gerar bigramas de todo o corpus
```

```
todos_bigramas = list(ngrams(todos_tokens, 2))
```

1.5 Frequência de N-gramas

```
# Contar frequência
```

```
freq_bigramas = FreqDist(todos_bigramas)
```

```
print("Bigramas mais frequentes:")
```

```
for bigrama, contagem in freq_bigramas.most_common(5):
```

```
    print(f"    {bigrama} → {contagem} vez(es)")
```


PARTE 2 — Modelos de Linguagem

2.1 O que é um Modelo de Linguagem?

Um modelo de linguagem é um sistema que atribui uma probabilidade a uma sequência de palavras. Ele responde à pergunta:

"Qual é a chance de essa sequência de palavras aparecer em um texto real?"

2.1 O que é um Modelo de Linguagem?

Exemplo

Qual das frases abaixo é mais provável em português?

Frase A: "o gato sentou no tapete"

Frase B: "tapete no sentou gato o"

Qualquer falante de português responderia imediatamente: a Frase A. Um modelo de linguagem quantifica essa intuição em números.

2.1 O que é um Modelo de Linguagem?

Para que serve um modelo de linguagem?

Aplicação	Como o modelo de linguagem ajuda
Autocomplete	Prevê a próxima palavra enquanto você digita
Correção ortográfica	Escolhe a palavra correta pelo contexto
Tradução automática	Seleciona a tradução mais natural
Reconhecimento de voz	Desambigua palavras parecidas pelo contexto
Geração de texto	Produz sequências de palavras com coerência
ChatGPT / Claude	Gera respostas palavra por palavra

2.2 Probabilidade de uma Sequência de Palavras

A probabilidade de uma frase é o produto das probabilidades de cada palavra dado o contexto anterior. Usando a regra da cadeia de probabilidades:

$$\begin{aligned} &P(\text{"o gato comeu o rato"}) \\ &= P(\text{"o"}) \\ &\times P(\text{"gato"} \mid \text{"o"}) \\ &\times P(\text{"comeu"} \mid \text{"o gato"}) \\ &\times P(\text{"o"} \mid \text{"o gato comeu"}) \\ &\times P(\text{"rato"} \mid \text{"o gato comeu o"}) \end{aligned}$$

2.2 Probabilidade de uma Sequência de Palavras

O problema de longo alcance

Calcular $P(\text{"rato"} \mid \text{"o gato comeu o"})$ exige ter visto essa exata sequência de 4 palavras antes. Em qualquer corpus finito, **a maioria dessas sequências longas nunca aparece — o que tornaria a probabilidade sempre zero.**

2.2 Probabilidade de uma Sequência de Palavras

A solução dos N-gramas: a hipótese de Markov

Os modelos N-gram simplificam o problema assumindo que a probabilidade de uma palavra depende apenas das $n-1$ palavras anteriores — não de toda a história:

Modelo unigrama: $P(\text{palavra})$ independe do contexto

Modelo bigrama: $P(\text{palavra} \mid \text{palavra anterior})$

Modelo trigrama: $P(\text{palavra} \mid \text{duas palavras anteriores})$

Essa simplificação é chamada de **hipótese de Markov**.

2.3 O Modelo Unigrama

O modelo mais simples: a probabilidade de cada palavra é calculada independentemente das demais, baseada apenas em sua frequência no corpus.

$$P(\text{palavra}) = \text{contagem}(\text{palavra}) / \text{total de palavras no corpus}$$

2.3 O Modelo Unigrama

Exemplo

Corpus: "o gato dorme o gato come o rato"

Total de tokens: 8

Palavra	Contagem	Probabilidade
o	3	$3/8 = 0,375$
gato	2	$2/8 = 0,250$
dorme	1	$1/8 = 0,125$
come	1	$1/8 = 0,125$
rato	1	$1/8 = 0,125$

2.3 O Modelo Unigrama

Probabilidade da frase "o gato dorme":

$$\begin{aligned}P(\text{"o gato dorme"}) &= P(\text{"o"}) \times P(\text{"gato"}) \times P(\text{"dorme"}) \\&= 0,375 \times 0,250 \times 0,125 \\&= 0,0117\end{aligned}$$

2.3 O Modelo Unigrama

```
from nltk.tokenize import word_tokenize
```

```
from nltk.probability import FreqDist
```

```
corpus = "o gato dorme o gato come o rato"
```

```
tokens = word_tokenize(corpus.lower(), language='portuguese')
```

```
freq = FreqDist(tokens)
```

```
total = len(tokens)
```


2.3 O Modelo Unigrama

```
# Probabilidade de cada palavra
```

```
print("Probabilidades (modelo unigrama):")
```

```
for palavra in freq:
```

```
    prob = freq[palavra] / total
```

```
    print(f"    P('{palavra}') = {freq[palavra]}/{total} = {prob:.4f}")
```

```
# Probabilidade de uma frase
```

```
frase_teste = ["o", "gato", "dorme"]
```

```
probabilidade = 1.0
```

2.3 O Modelo Unigrama

```
for palavra in frase_teste:
    prob_palavra = freq[palavra] / total
    probabilidade = probabilidade * prob_palavra

print(f"\nP('o gato dorme') = {probabilidade:.6f}")
```

2.4 O Modelo Bigrama

O modelo bigrama calcula a probabilidade de cada palavra dado a palavra anterior.

$$P(\text{palavra_atual} \mid \text{palavra_anterior}) = \text{contagem}(\text{palavra_anterior}, \text{palavra_atual}) / \text{contagem}(\text{palavra_anterior})$$

2.4 O Modelo Bigrama

Exemplo

Corpus: "o gato dorme o gato come o rato"

Bigramas presentes:

("o", "gato") → 2 vezes

("gato", "dorme") → 1 vez

("dorme", "o") → 1 vez

("gato", "come") → 1 vez

("come", "o") → 1 vez

("o", "rato") → 1 vez

2.4 O Modelo Bigrama

Contagem das palavras que iniciam bigramas:

"o" → 3 vezes (inicia "o gato" x2 e "o rato" x1)

"gato" → 2 vezes (inicia "gato dorme" e "gato come")

2.4 O Modelo Bigrama

Calculando probabilidades condicionais:

$$P(\text{"gato"} \mid \text{"o"}) = \text{contagem}(\text{"o gato"}) / \text{contagem}(\text{"o"}) = 2/3 = 0,667$$

$$P(\text{"dorme"} \mid \text{"gato"}) = \text{contagem}(\text{"gato dorme"}) / \text{contagem}(\text{"gato"}) = 1/2 = 0,500$$

$$P(\text{"come"} \mid \text{"gato"}) = \text{contagem}(\text{"gato come"}) / \text{contagem}(\text{"gato"}) = 1/2 = 0,500$$

$$P(\text{"rato"} \mid \text{"o"}) = \text{contagem}(\text{"o rato"}) / \text{contagem}(\text{"o"}) = 1/3 = 0,333$$

2.4 O Modelo Bigrama

Probabilidade da frase "o gato dorme" com modelo bigrama:

$$\begin{aligned}P(\text{"o gato dorme"}) &= P(\text{"o"}) \times P(\text{"gato"} | \text{"o"}) \times P(\text{"dorme"} | \text{"gato"}) \\&= (3/8) \times (2/3) \times (1/2) \\&= 0,375 \times 0,667 \times 0,500 \\&= 0,125\end{aligned}$$

**Observe que o modelo bigrama deu probabilidade mais alta do que o unigrama (0,125 vs. 0,0117)
porque leva o contexto em conta.**

2.4 O Modelo Bigrama

```
from nltk import ngrams

from nltk.tokenize import word_tokenize

from nltk.probability import FreqDist

corpus = "o gato dorme o gato come o rato"

tokens = word_tokenize(corpus.lower(), language='portuguese')

# Contar unigramas e bigramas

freq_uni = FreqDist(tokens)

freq_bi = FreqDist(ngrams(tokens, 2))
```


2.4 O Modelo Bigrama

```
# Calcular probabilidade condicional de bigramas

print("Probabilidades condicionais (modelo bigrama):")

print()

for bigrama, contagem in freq_bi.items():

    palavra_ant = bigrama[0]

    palavra_atu = bigrama[1]

    prob = contagem / freq_uni[palavra_ant]

    print(f"                P('{palavra_atu}'          |          '{palavra_ant}')
          {contagem}/{freq_uni[palavra_ant]} = {prob:.4f}")
```

2.4 O Modelo Bigrama

```
# Probabilidade da frase "o gato dorme"
frase_teste = ["o", "gato", "dorme"]

prob_frase = freq_uni[frase_teste[0]] / len(tokens) # P("o") – unigrama inicial

for i in range(1, len(frase_teste)):
    ant      = frase_teste[i-1]
    atu      = frase_teste[i]
    bigrama  = (ant, atu)
    prob_cond = freq_bi[bigrama] / freq_uni[ant]
    prob_frase = prob_frase * prob_cond

print(f"\nP('o gato dorme') com bigrama = {prob_frase:.6f}")
```

2.5 O Problema das Probabilidades Zero

O modelo bigrama tem um problema grave: se uma sequência de palavras nunca apareceu no corpus de treino, **sua probabilidade será zero.**

E uma probabilidade zero numa multiplicação contamina tudo — **o produto inteiro vira zero.**

2.5 O Problema das Probabilidades Zero

Exemplo

Corpus de treino: "o gato dorme o gato come"

Frase nova para avaliar: "o gato late"

$$\begin{aligned} P(\text{"late"} \mid \text{"gato"}) &= \text{contagem}(\text{"gato late"}) / \text{contagem}(\text{"gato"}) \\ &= 0 / 2 \\ &= 0 \end{aligned}$$

Resultado: $P(\text{"o gato late"}) = 0$

Isso não está certo. O modelo não deveria dizer que é impossível um gato fazer barulho — **ele simplesmente não viu esse bigrama no treino.**

PARTE 3 — Suavização de Laplace

3.1 O que é suavização?

A suavização (ou *smoothing*) é uma técnica que redistribui uma pequena probabilidade para eventos que nunca foram observados no corpus de treino.

A ideia é simples: em vez de dar probabilidade zero para o que nunca foi visto, dar um valor muito pequeno — mas positivo.

3.2 Suavização de Laplace (Add-1)

A suavização mais simples — proposta por Pierre-Simon Laplace no século XVIII — consiste em somar 1 à contagem de todos os bigramas, incluindo os que nunca apareceram.

Fórmula com suavização de Laplace

$$P_{\text{Laplace}}(\text{palavra_atual} \mid \text{palavra_anterior}) = \frac{\text{contagem}(\text{palavra_anterior}, \text{palavra_atual}) + 1}{\text{contagem}(\text{palavra_anterior}) + V}$$

Onde **V** é o tamanho do vocabulário (número de palavras únicas).

3.2 Suavização de Laplace (Add-1)

Por que somar V no denominador?

Se somamos 1 à contagem de todos os V bigramas possíveis, precisamos compensar no denominador para que as probabilidades ainda somem 1 (distribuição válida). Exemplo:

Corpus: "o gato dorme o gato come"

Vocabulário: {o, gato, dorme, come} $\rightarrow V = 4$

3.2 Suavização de Laplace (Add-1)

Sem suavização:

$$P(\text{"late"} \mid \text{"gato"}) = 0/2 = 0,000 \quad \leftarrow \text{problema!}$$

Com suavização de Laplace:

$$P(\text{"late"} \mid \text{"gato"}) = (0 + 1) / (2 + 4) = 1/6 = 0,167$$

$$P(\text{"dorme"} \mid \text{"gato"}) = (1 + 1) / (2 + 4) = 2/6 = 0,333$$

$$P(\text{"come"} \mid \text{"gato"}) = (1 + 1) / (2 + 4) = 2/6 = 0,333$$

Observe que a suavização redistribui a probabilidade. As palavras conhecidas perdem um pouco de probabilidade para dar espaço às palavras nunca vistas. Isso é o "**custo**" da suavização.

3.2 Suavização de Laplace (Add-1)

```
from nltk import ngrams

from nltk.tokenize import word_tokenize

from nltk.probability import FreqDist

corpus = "o gato dorme o gato come o rato"

tokens = word_tokenize(corpus.lower(), language='portuguese')

freq_uni = FreqDist(tokens)

freq_bi = FreqDist(ngrams(tokens, 2))
```

3.2 Suavização de Laplace (Add-1)

```
V = len(freq_uni)    # tamanho do vocabulário

print(f"Vocabulário: {list(freq_uni.keys())}")
print(f"Tamanho do vocabulário (V): {V}")
print()

# Probabilidade com suavização de Laplace
def prob_laplace(palavra_ant, palavra_atu, freq_uni, freq_bi, V):
    bigrama      = (palavra_ant, palavra_atu)
    numerador    = freq_bi[bigrama] + 1
    denominador  = freq_uni[palavra_ant] + V
    return numerador / denominador
```


3.2 Suavização de Laplace (Add-1)

```
# Testar
pares = [
    ("gato", "dorme"),    # visto no treino
    ("gato", "come"),     # visto no treino
    ("gato", "late"),     # NUNCA visto – teste da suavização
    ("gato", "voa"),      # NUNCA visto – teste da suavização
]

print("Probabilidades com suavização de Laplace:")
for ant, atu in pares:
    p = prob_laplace(ant, atu, freq_uni, freq_bi, V)
    visto = "✓ visto no treino" if freq_bi[(ant, atu)] > 0 else "X nunca visto"
    print(f"  P('{atu}' | '{ant}') = {p:.4f}    {visto}")
```

3.3 Suavização Add-k

Uma variação da suavização de Laplace é somar k em vez de 1, onde k é um número menor que 1 (como 0,5 ou 0,1). Isso redistribui menos probabilidade para eventos não vistos, preservando mais a distribuição original:

$$P_{\text{Add-k}}(\text{palavra_atual} \mid \text{palavra_anterior}) = \frac{\text{contagem}(\text{palavra_anterior}, \text{palavra_atual}) + k}{\text{contagem}(\text{palavra_anterior}) + k \times V}$$

O valor de k é um hiperparâmetro que pode ser ajustado conforme o corpus.

PARTE 4 — Perplexidade

4.1 Como medir a qualidade de um modelo de linguagem?

Temos um modelo de linguagem treinado. Como sabemos se ele é bom?

Uma métrica muito usada é a **perplexidade**. Ela mede o quanto o modelo ficou "*confuso*" ao tentar prever um texto de teste.

Intuição: imagine que o modelo precisa adivinhar a próxima palavra. Se ele tem baixa perplexidade, estava "*confiante*" — suas previsões eram boas. Se tem alta perplexidade, estava "*confuso*" — as palavras do texto eram surpresas.

4.2 A fórmula da perplexidade

Para um texto de teste com N palavras:

$$\text{Perplexidade} = 2^{-(1/N) \times \text{soma dos } \log_2(P(\text{cada palavra}))}$$

Ou de forma equivalente, usando a probabilidade do texto inteiro:

$$\text{Perplexidade} = P(\text{texto inteiro})^{-1/N}$$

4.2 A fórmula da perplexidade

Interpretação

Perplexidade	Interpretação
Baixa (~1)	Modelo excelente — previu todas as palavras com certeza
Média (~100)	Modelo razoável
Alta (~1000+)	Modelo ruim — muitas surpresas

4.2 A fórmula da perplexidade

Exemplo

Um modelo unigrama avaliado em "o gato dorme" com probabilidades:

$$P(\text{"o"}) = 0,375$$

$$P(\text{"gato"}) = 0,250$$

$$P(\text{"dorme"}) = 0,125$$

Probabilidade da frase:

$$P(\text{"o gato dorme"}) = 0,375 \times 0,250 \times 0,125 = 0,01172$$

4.2 A fórmula da perplexidade

Perplexidade (**N=3** palavras):

$$\begin{aligned}\text{Perplexidade} &= (0,01172)^{-1/3} \\ &= (0,01172)^{-0,333} \\ &\approx 4,16\end{aligned}$$

Isso significa que o modelo estava, em média, tão incerto quanto se tivesse que escolher entre ~4 palavras igualmente prováveis a cada passo.

4.2 A fórmula da perplexidade

```
import math

# Probabilidades do modelo unigrama para o texto de teste
probs_texto = [0.375, 0.250, 0.125] # P("o"), P("gato"), P("dorme")
N = len(probs_texto)

# Calcular a probabilidade total (produto)
prob_total = 1.0
for p in probs_texto:
    prob_total = prob_total * p

print(f"Probabilidade do texto: {prob_total:.6f}")
```

4.2 A fórmula da perplexidade

```
# Calcular a perplexidade
perplexidade = prob_total ** (-1/N)

print(f"Perplexidade: {perplexidade:.4f}")
print()
print("Interpretação:")
print(f"    O modelo estava tão incerto quanto escolher entre
~{perplexidade:.1f} palavras")
print(f" a cada passo. Quanto menor, melhor o modelo.")
```


4.3 Por que usar logaritmos?

Na prática, multiplicar muitas probabilidades pequenas causa um problema: o número fica tão pequeno que o computador o arredonda para zero (chamado de *underflow* numérico).

A solução é somar os logaritmos das probabilidades em vez de multiplicá-las:

$$\log(P(w1) \times P(w2) \times P(w3)) = \log(P(w1)) + \log(P(w2)) + \log(P(w3))$$

Isso funciona porque logaritmo transforma multiplicação em adição.

4.3 Por que usar logaritmos?

```
import math

probs_texto = [0.375, 0.250, 0.125]
N = len(probs_texto)

# Somar logaritmos em vez de multiplicar probabilidades
soma_logs = 0.0
for p in probs_texto:
    soma_logs = soma_logs + math.log2(p)

# Perplexidade com logaritmos
perplexidade = 2 ** (-soma_logs / N)

print(f"Soma dos log2: {soma_logs:.4f}")
print(f"Perplexidade: {perplexidade:.4f}")
```

PARTE 5 — Geração de Texto com N-gramas

5.1 Como um modelo N-gram gera texto?

Um modelo de linguagem N-gram pode ser usado para **gerar texto** de forma simples:

1. Escolher uma palavra inicial (ou par de palavras)
2. Olhar quais palavras costumam aparecer depois dessa palavra no corpus
3. Escolher uma das opções com probabilidade proporcional à frequência
4. Repetir o processo até completar o texto

Esse processo é chamado de **amostragem** (*sampling*).

5.2 Geração com modelo bigrama

```
import random

from nltk import ngrams
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist

# Corpus de treino
corpus = """
o gato dorme no sofá o gato come o rato o gato brinca com o rato
o rato corre do gato o rato dorme no chão o rato come queijo
"""

tokens = word_tokenize(corpus.lower(), language='portuguese')

# Contar unigramas e bigramas
freq_uni = FreqDist(tokens)
freq_bi = FreqDist(ngrams(tokens, 2))
```

5.2 Geração com modelo bigrama

Montar dicionário: palavra → lista de próximas palavras (com repetição)

```
proximo = {}
```

```
for bigrama in freq_bi:
```

```
    palavra_ant = bigrama[0]
```

```
    palavra_atu = bigrama[1]
```

```
    if palavra_ant not in proximo:
```

```
        proximo[palavra_ant] = []
```

```
# Adicionar à lista tantas vezes quanto a frequência
```

```
for i in range(freq_bi[bigrama]):
```

```
    proximo[palavra_ant].append(palavra_atu)
```


5.2 Geração com modelo bigrama

```
# Gerar texto

palavra_atual = "o"    # palavra inicial
texto_gerado  = [palavra_atual]

for passo in range(9):    # gerar mais 9 palavras
    if palavra_atual in proximo:
        opcoes      = proximo[palavra_atual]
        proxima      = random.choice(opcoes)    # escolher aleatoriamente
        texto_gerado.append(proxima)
        palavra_atual = proxima
    else:
        break    # parar se não houver continuação

print("Texto gerado:")
print(" ".join(texto_gerado))
```

5.3 Limitações dos modelos N-gram para geração

Limitação	Descrição
Contexto curto	Um bigrama só lembra a palavra anterior — esquece o início da frase
Incoerência	O texto gerado pode não fazer sentido globalmente
Vocabulário fechado	Palavras fora do vocabulário de treino causam probabilidade zero
Não capta semântica	Não entende o significado das palavras — apenas padrões de co-ocorrência

Essas limitações motivaram o desenvolvimento de modelos neurais como as RNNs, LSTMs e, mais recentemente, os Transformers (Semana 11).

PARTE 6 — Escolhendo o Valor de N

6.1 Trade-off entre N e qualidade

A escolha do valor de N envolve um equilíbrio entre dois problemas opostos:

Problema	Quando ocorre	Consequência
Underfitting	N muito pequeno (n=1)	Contexto insuficiente — modelo muito genérico
Esparsidade	N muito grande (n=5+)	Sequências longas raramente aparecem — probabilidade zero em quase tudo

6.1 Trade-off entre N e qualidade

Por que $N=1$ é insuficiente?

Com unigramas, a probabilidade de cada palavra é independente do contexto. O modelo não sabe que depois de "aprendizado" é mais provável "de" do que "elefante".

Por que N muito grande é problemático?

Com 5-gramas, o modelo precisa ter visto a sequência exata de 5 palavras no treino. Em qualquer corpus finito, a maioria das sequências longas nunca apareceu — gerando probabilidade zero na maioria dos casos.

6.2 Valores típicos na prática

Valor de N	Quando usar
n=1 (unigrama)	Apenas como baseline — muito fraco
n=2 (bigrama)	Corpus pequeno, tarefas simples
n=3 (trigrama)	Bom equilíbrio para corpus médio
n=4 ou 5	Corpus muito grande (bilhões de palavras)
n=1 a 2 em BoW/TF-IDF	Features para ML — padrão da indústria

PARTE 7 — N-gramas em PLN Moderno

7.1 N-gramas como features no TF-IDF

A aplicação mais prática dos n-gramas no contexto desta disciplina é como features adicionais no CountVectorizer e TfidfVectorizer do scikit-learn:

7.1 N-gramas como features no TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
reviews = [  
    "produto bom não ruim",  
    "não funciona produto ruim",  
    "produto excelente qualidade boa",  
]
```

7.1 N-gramas como features no TF-IDF

```
# Unigramas apenas (padrão)
```

```
tfidf_uni = TfidfVectorizer(ngram_range=(1, 1))
```

```
X_uni = tfidf_uni.fit_transform(reviews)
```

```
print("Vocab. unigramas:", tfidf_uni.get_feature_names_out().tolist())
```

```
# Unigramas + Bigramas
```

```
tfidf_bi = TfidfVectorizer(ngram_range=(1, 2))
```

```
X_bi = tfidf_bi.fit_transform(reviews)
```

```
print("Vocab. uni+bi: ", tfidf_bi.get_feature_names_out().tolist())
```


7.2 A relação com os modelos modernos

Os modelos N-gram são a base histórica e conceitual de toda a evolução do PLN:

Modelos N-gram (1990–2012)



Word Embeddings – Word2Vec, GloVe (2013+)



Modelos Neurais – RNN, LSTM (2015+)



Transformers – BERT, GPT (2017+)

Os Transformers de hoje (ChatGPT, Claude) fazem essencialmente o mesmo trabalho de um modelo de linguagem N-gram — calcular a probabilidade da próxima palavra — mas com uma capacidade muito maior de capturar contexto longo e semântica.

PARTE 8 — Fórmulas essenciais

Modelo unigrama

$$P(w) = \text{contagem}(w) / \text{total de tokens}$$

Modelo bigrama

$$P(w_2 \mid w_1) = \text{contagem}(w_1, w_2) / \text{contagem}(w_1)$$

Bigrama com suavização de Laplace

$$P(w_2 \mid w_1) = (\text{contagem}(w_1, w_2) + 1) / (\text{contagem}(w_1) + V)$$

Perplexidade (versão com logaritmo)

$$PP = 2^{-(1/N) \times \sum \log_2(P(w_i))}$$

Referências

CASELI, H. M.; NUNES, M. G. V. (org.). Processamento de linguagem natural: conceitos, técnicas e aplicações em português. São Carlos: BPLN, 2023. Cap. 7, pp. 181-214.

FACELI, K. et al. Inteligência Artificial: uma abordagem de aprendizado de máquina. Rio de Janeiro: LTC, 2025. Cap. 8.

MARTINS, J. S. et al. Processamento de Linguagem Natural. Porto Alegre: Grupo A, 2024. Cap. 4, pp. 73–98.

Recursos online gratuitos

- **NLTK Book Cap. 2:** nltk.org/book/ch02.html — Corpora e FreqDist
- **Speech and Language Processing — Jurafsky & Martin (2024):**
web.stanford.edu/~jurafsky/slp3/ — Capítulo 3: N-gram Language Models (gratuito online)



IBMEC.BR

 /IBMEC

 IBMEC

 @IBMEC_OFICIAL

 @IBMEC

 **ibmec**